



北京城市学院信息学部

20252026 学年秋季学期

工程二级实训（全栈开发）期末考核报告

专 业： _____ 软件工程 _____

班 级： _____ 23 软本 1 _____

姓 名： _____ 郭昕瑞 _____

2025 年 12 月

目 录

一、 需求分析	1
(一) 问题描述	1
(二) 基本要求	1
二、 详细设计	2
(一) 后端 API 设计	2
1. 登录 API 实现	2
2. 商品分类管理 API 实现	6
3. 商品管理 API 实现	11
4. 用户管理 API 实现	19
(二) 前端 Vue 管理端	26
1. 登录页面	26
2. 商品分类管理页面	30
3. 商品管理页面	37
4. 用户管理页面	51
三. 收获和体会	61

基于全栈技术的商品展示系统开发实践

一、需求分析

（一）问题描述

本系统是一个基于全栈技术的商品展示管理系统，采用前后端分离架构。系统主要实现商品信息的增删改查、商品分类管理、用户管理等功能。

使用的技术栈：

后端技术：

Node.js + Express：构建 RESTful API 服务

MySQL：关系型数据库存储

ORM2：对象关系映射框架，简化数据库操作

Passport.js：用户认证中间件

JWT (jsonwebtoken)：基于 Token 的身份验证

Multer：文件上传中间件

Bluebird：Promise 库，处理异步操作

前端技术：

Vue 3：渐进式 JavaScript 框架

Vue Router：路由管理

Vuex：状态管理

Element Plus：UI 组件库

Axios：HTTP 客户端

Vue Quill：富文本编辑器

系统架构：

前端 Vue 单页应用，通过 Axios 调用后端 API

后端 Express 服务器，提供 RESTful API 接口

MySQL 数据库存储数据

JWT Token 实现用户身份认证

（二）基本要求

1. 用户登录功能

支持管理员用户名和密码登录

登录成功后返回 JWT Token

Token 存储在 localStorage 中，后续请求携带 Token 进行身份验证

2. 商品分类管理功能

查看商品分类列表

添加商品分类（支持多级分类）

编辑商品分类信息

删除商品分类

支持按分类名称搜索

3. 商品管理功能

- 查看商品列表（支持分页）
- 按商品名称搜索
- 按商品分类筛选
- 添加商品（包含商品名称、价格、重量、数量、分类、图片、详情）
- 编辑商品信息
- 删除商品
- 支持商品图片上传

4. 用户管理功能

- 查看用户列表（支持分页）
- 按用户名搜索
- 查看用户详细信息
- 编辑用户信息（用户名、密码、手机、邮箱、角色、状态）
- 删除用户
- 启用/禁用用户状态切换

5. 系统要求

- 所有接口（除登录外）都需要 Token 认证
- 支持跨域访问
- 数据验证和错误处理
- 友好的用户界面和交互体验

二、详细设计

（一）后端 API 设计

1. 登录 API 及数据库设计实现

（1）登录 API 实现过程描述

登录 API 采用 Passport.js 的 Local 策略进行身份验证，实现流程如下：

步骤 1：配置 Passport 中间件 在 modules/passport.js 中配置 Passport，使用 passportlocal 策略：

```
passport.use(new LocalStrategy({
  usernameField: 'username',
  passwordField: 'password'
}, ManagerService.login));
```

步骤 2：实现登录服务 在 services/ManagerService.js 中实现登录验证逻辑：

- 根据用户名查询数据库中的用户信息
- 验证用户是否存在
- 检查用户角色权限 (role_id >= 0)

检查用户状态 (mg_state = 1 表示启用)

验证密码是否匹配

验证成功返回用户信息，失败返回错误信息

步骤 3: 生成 JWT Token 登录成功后，使用 jsonwebtoken 生成 Token:

```
const token = jwt.sign(  
  { id: user.id, rid: user.rid },  
  config.secret,  
  { expiresIn: '24h' }  
);
```

步骤 4: 设置路由 在 app.js 中配置登录路由:

```
app.use('/sysapi/login',  
sys_passport.login);
```

步骤 5: Token 认证中间件 使用 passporthttpbearer 策略验证 Token:

```
app.use('/sysapi/users',  
sys_passport.tokenAuth);
```

数据流向:

前端发送登录请求 → Passport 验证 →
ManagerService 查询数据库 →
验证用户信息 → 生成 JWT Token → 返回
Token 给前端

(2) 登录 API 接口测试截图

数据库连接配置:

在 config/default.json 中配置数据库连接信息:

```
{
  "db": {
    "host": "localhost",
    "database": "fruitshop",
    "user": "root",
    "password": "123456",
    "port": 3306
  }
}
```

数据库初始化:

在 modules/db.js 中实现数据库连接和模型初始化:

```
orm.connect(settings, function(err,
db) {
  if(err) throw err;
  // 加载数据模型
  require('../models/ManagerModel')
  (db);
  require('../models/CategoryModel')
  (db);
  require('../models/GoodModel')
  (db);
  require('../models/GoodPicModel')
  (db);
});
```

数据模型定义:

在 models/ManagerModel.js 中定义管理员模型:

```
module.exports = function(db,
callback) {
  db.define("ManagerModel", {
    mg_id: {type: 'serial', key:
      true},
```

```

        mg_name: String,
        mg_pwd: String,
        mg_time: Number,
        role_id: Number,
        mg_mobile: String,
        mg_email: String,
        mg_state: Number
    }, {table: "manager"}));
    return callback();
};

```

2. 商品分类管理 API 实现

(1) 商品分类管理 API 实现过程描述

商品分类管理 API 采用分层架构设计，包括路由层、服务层、DAO 层。

路由层设计：

在 routes/sysapi/category.js 中定义分类管理路由：

```

GET / 获取分类列表
GET /:id 获取分类详情
POST / 添加分类
PUT /:id 更新分类
DELETE /:id 删除分类

```

每个路由都包含参数验证中间件和业务逻辑处理。

服务层实现：

在 services/CategoryService.js 中实现业务逻辑：

```

getAllCategories() 获取所有分类，支持关键词搜索
getCategoryById() 根据 ID 获取分类详情
createCategory() 创建新分类，验证分类名称唯一性
updateCategory() 更新分类信息
deleteCategory() 删除分类

```

DAO 层封装：

使用通用 DAO 模块 dao/DAO.js 进行数据库操作：

```

findOne() 查询单条记录
list() 查询多条记录
create() 创建记录
update() 更新记录

```


数据模型:

在 models/CategoryModel.js 中定义分类模型:

```
module.exports = function(db,
callback) {
  db.define("CategoryModel", {
    cat_id: {type: 'serial', key:
      true},
    cat_name: String,
    cat_pid: Number,
    cat_level: Number
  }, {table: "category"});
  return callback();
};
```

实现流程:

前端请求 → 路由参数验证 → 服务层业务逻辑
→ DAO 层数据库操作 → 返回结果

(2) 商品分类管理 API 接口测试截图

API Network

POST http://localhost:8088/... GET http://127.0.0.1:8088/sy/... No environment

http://127.0.0.1:8088/sysapi/category

GET http://127.0.0.1:8088/sysapi/category Send

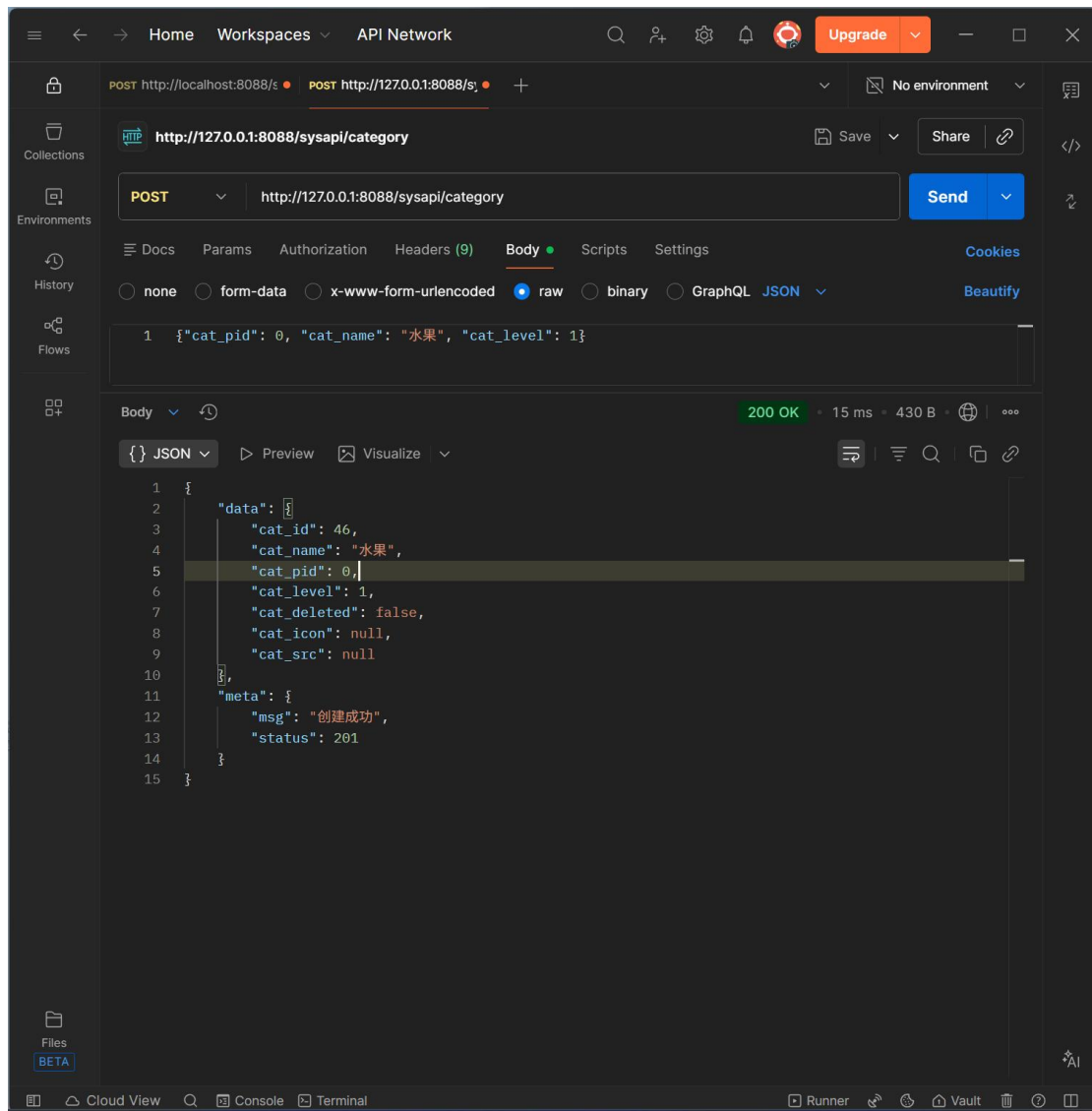
Docs Params Authorization Headers (7) Body Scripts Settings Cookies

Body

200 OK 9 ms 1 KB

JSON Preview Visualize

```
1 {
2   "data": [
3     {
4       "cat_id": 11,
5       "cat_name": "奇异果",
6       "cat_pid": 0,
7       "cat_level": 0,
8       "cat_deleted": false,
9       "cat_icon": null,
10      "cat_src": null
11    },
12    {
13      "cat_id": 12,
14      "cat_name": "葡萄提莓果",
15      "cat_pid": 0,
16      "cat_level": 0,
17      "cat_deleted": false,
18      "cat_icon": null,
19      "cat_src": null
20    },
21    {
22      "cat_id": 13,
23      "cat_name": "柑橘橙柚",
24      "cat_pid": 0,
25      "cat_level": 0,
26      "cat_deleted": false,
27      "cat_icon": null,
28      "cat_src": null
29    },
30    {
31      "cat_id": 14,
32      "cat_name": "苹果蕉梨",
33      "cat_pid": 0,
34      "cat_level": 0,
35      "cat_deleted": false,
```



Home Workspaces API Network

POST http://localhost:8088/ PUT http://127.0.0.1:8088/sy. +

http://127.0.0.1:8088/sysapi/category/5

PUT http://127.0.0.1:8088/sysapi/category/5

Send

Docs Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beautify

```
1 {"cat_name": "新鲜水果"}
```

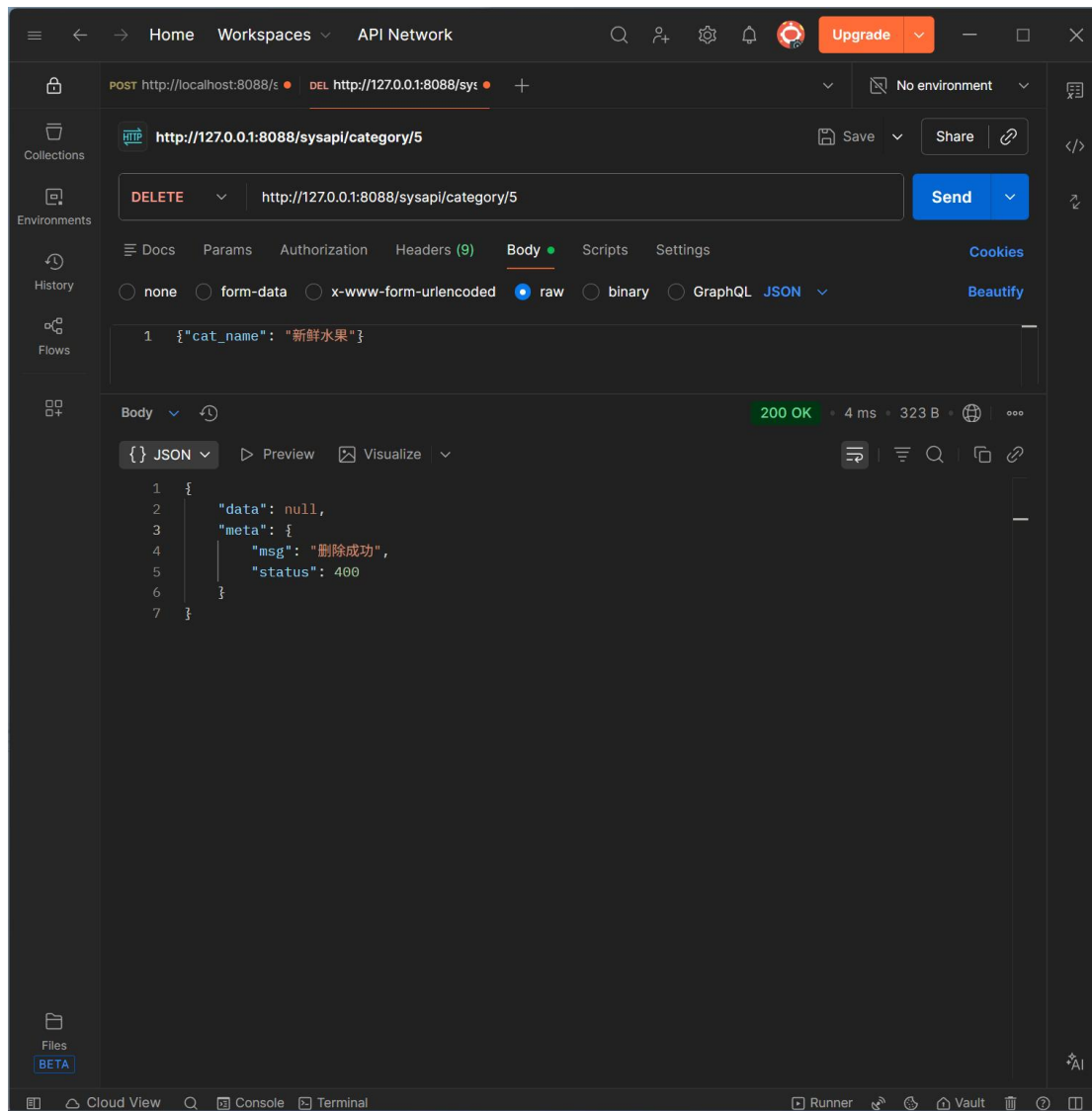
Body 200 OK 17 ms 434 B

JSON Preview Visualize

```
1 {
2   "data": {
3     "cat_id": 5,
4     "cat_name": "新鲜水果",
5     "cat_pid": 0,
6     "cat_level": 0,
7     "cat_deleted": true,
8     "cat_icon": null,
9     "cat_src": null
10  },
11  "meta": {
12    "msg": "更新成功",
13    "status": 200
14  }
15 }
```

Files BETA

Cloud View Console Terminal Runner Vault



3. 商品管理 API 实现

(1) 商品管理 API 实现过程描述

商品管理 API 实现了商品的完整 CRUD 操作，包括商品基本信息、商品图片、商品详情的管理。

路由层设计：

在 routes/sysapi/goods.js 中定义商品管理路由：

GET / 获取商品列表（支持分页、搜索、分类筛选）

GET /:id 获取商品详情

POST / 添加商品

PUT /:id 更新商品

DELETE /:id 删除商品

服务层实现：

在 services/GoodService.js 中实现业务逻辑：

getAllGoods() 获取商品列表，支持分页、搜索、分类筛选
getGoodsById() 获取商品详情（包含图片信息）
createGood() 创建商品，处理商品图片关联
updateGood() 更新商品信息
deleteGood() 删除商品
分页查询实现：

```
module.exports.getAllGoods =
function(params, cb) {
    var conditions = {};
    if (params.query) {
        conditions['columns']
        ['goods_name'] = orm.like('%' +
        params.query + '%');
    }
    if (params.cat_id) {
        conditions['columns']
        ['goods_cat'] = params.cat_id;
    }

    dao.countByConditions
    ('GoodModel', conditions, function
    (err, count) {
        var offset = (params.pagenum
        1) * params.pagesize;
        conditions['offset'] = offset;
        conditions['limit'] = params.
        pagesize;
        conditions['order'] =
        'goods_id';

        dao.list('GoodModel',
        conditions, function(err,
        goods) {
            cb(null, {total: count,
            goods: goods});
        });
    });
};
```

数据模型：

商品模型 (models/GoodModel.js) :

```
module.exports = function(db,
callback) {
  db.define("GoodModel", {
    goods_id: {type: 'serial', key:
      true},
    goods_name: String,
    goods_price: Number,
    goods_number: Number,
    goods_weight: Number,
    goods_cat: String,
    goods_introduce: String,
    goods_big_logo: String,
    goods_small_logo: String,
    goods_state: Number
  }, {table: "goods"});
  return callback();
};
```

商品图片模型 (models/GoodPicModel.js) :

```
module.exports = function(db,
callback) {
  db.define("GoodPicModel", {
    pics_id: {type: 'serial', key:
      true},
    goods_id: Number,
    pics_big: String,
    pics_mid: String,
    pics_sma: String,
    pics_name: String
  }, {table: "goodspics"});
  return callback();
};
```

文件上传实现:

在 routes/sysapi/upload.js 中使用 Multer 处理文件上传:

```
var storage = multer.diskStorage({
  destination: function(req, file,
    cb) {
    cb(null, 'tmp_uploads/');
  },
  filename: function(req, file, cb)
```

```
    {
      cb(null, Date.now() + path.
        extname(file.originalname));
    }
  });

var upload = multer({storage:
  storage});

router.post('/', upload.single
  ('file'), function(req, res) {
  res.sendResult({
    tmp_path: req.file.filename
  }, 200, '上传成功');
});
```

(2) 商品管理 API 接口测试截图

API Network interface showing a GET request to `http://127.0.0.1:8088/sysapi/goods` with headers and a JSON response.

Request Details:

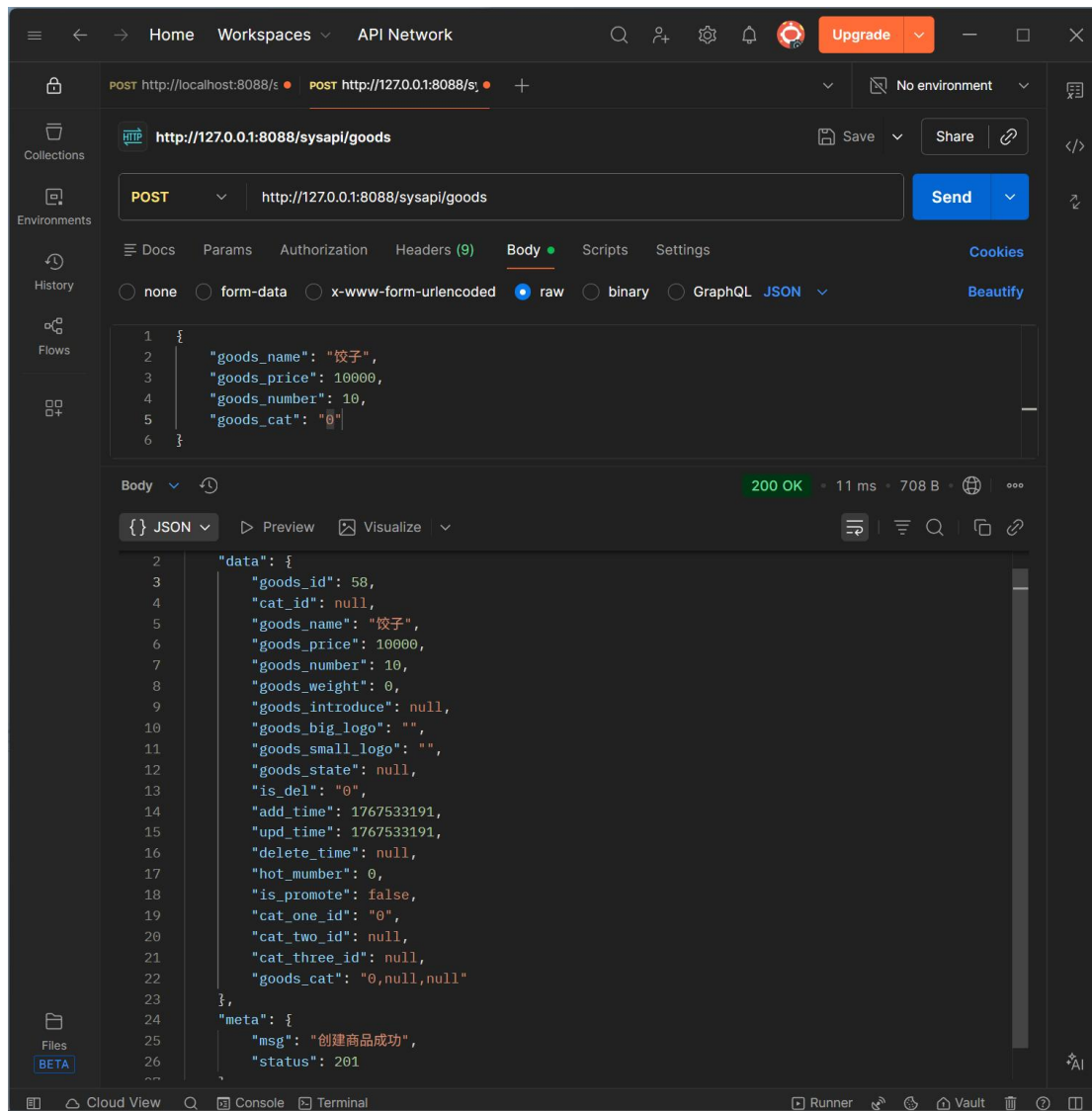
- Method: GET
- URL: `http://127.0.0.1:8088/sysapi/goods?pagenum=1&pagesize=10`
- Headers (9):
 - Connection: keep-alive
 - Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6I...

Response Details:

- Status: 200 OK
- Time: 5 ms
- Size: 1.85 KB

JSON Response Body:

```
{
  "hot_mumber": 0,
  "is_promote": false,
  "cat_one_id": null,
  "cat_two_id": null,
  "cat_three_id": null
},
{
  "goods_id": 13,
  "cat_id": 0,
  "goods_name": "妃子笑荔枝",
  "goods_price": 99.8,
  "goods_number": 100,
  "goods_weight": 1,
  "goods_state": 0,
  "add_time": 1683887715,
  "upd_time": 1683887715,
  "hot_mumber": 0,
  "is_promote": false,
  "cat_one_id": null,
  "cat_two_id": null,
  "cat_three_id": null
}
{
  "meta": {
    "msg": "获取成功",
    "status": 200
  }
}
```



API Network interface showing a PUT request to `http://127.0.0.1:8088/sysapi/goods/58`.

Request:

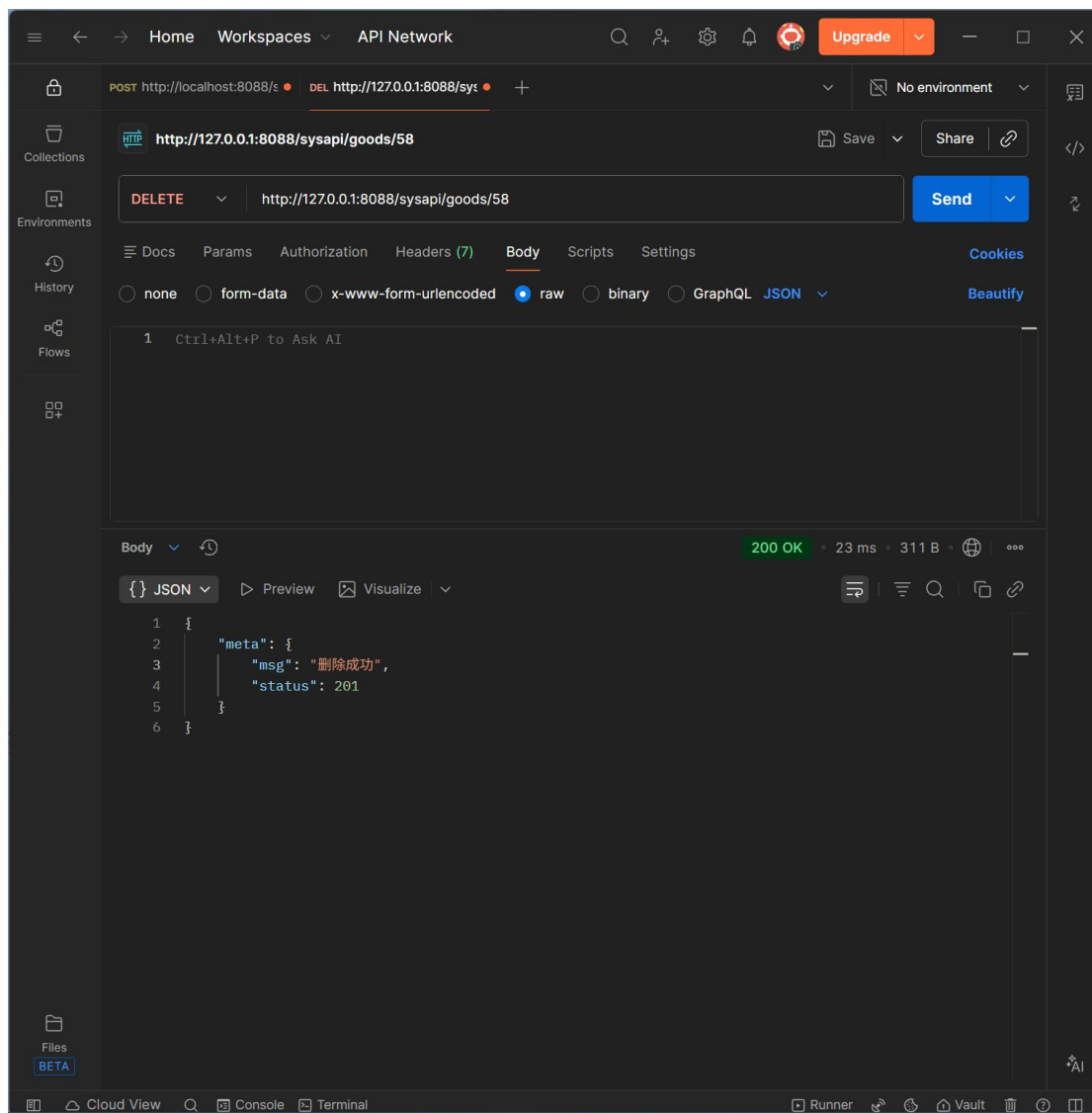
- Method: PUT
- URL: `http://127.0.0.1:8088/sysapi/goods/58`
- Body (raw):

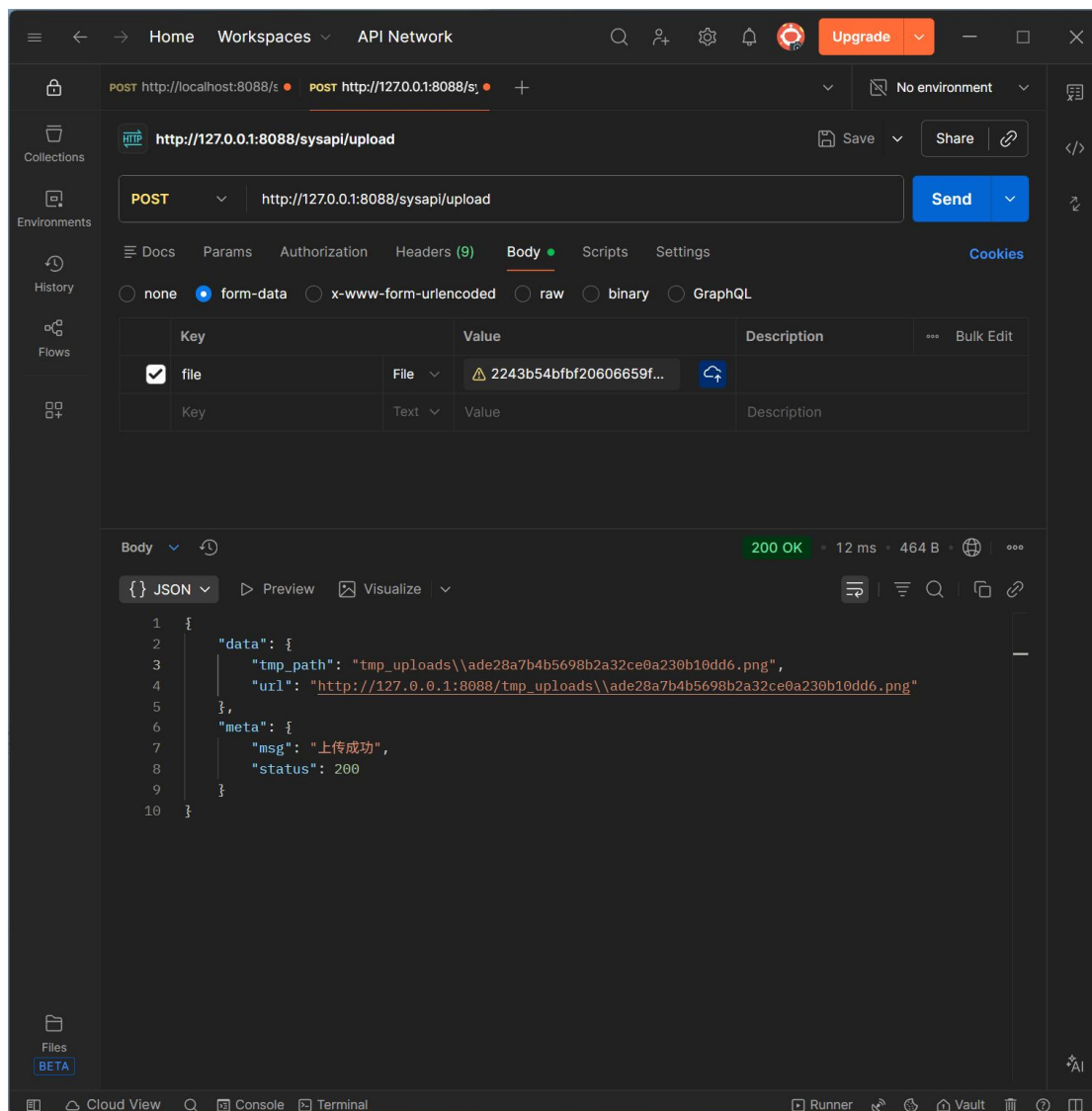
```
1 {
2   "goods_name": "饺子不好吃",
3   "goods_price": 10000,
4   "goods_number": 10,
5   "goods_cat": "0"
6 }
```

Response:

- Status: 200 OK
- Time: 13 ms
- Size: 689 B
- Body (JSON):

```
0 goods_price: 10000,
1 "goods_number": 10,
2 "goods_weight": 0,
3 "goods_introduce": null,
4 "goods_big_logo": "",
5 "goods_small_logo": "",
6 "goods_state": 0,
7 "is_del": "0",
8 "add_time": 1767533311,
9 "upd_time": 1767533311,
10 "delete_time": null,
11 "hot_mumber": 0,
12 "is_promote": false,
13 "cat_one_id": "0",
14 "cat_two_id": 0,
15 "cat_three_id": 0,
16 "pics": []
17 },
18 "meta": {
19   "msg": "更新商品成功",
20   "status": 200
21 }
22 }
```





4. 用户管理 API 实现

(1) 用户管理 API 实现过程描述

用户管理 API 实现了管理员用户的增删改查功能，包括用户状态管理。

路由层设计：

在 routes/sysapi/users.js 中定义用户管理路由：

GET / 获取用户列表（支持分页、搜索）

GET /:id 获取用户详情

PUT /:id 更新用户信息

DELETE /:id 删除用户（软删除）

参数验证中间件：

```
router.get('/', function(req, res,
next) {
```

```

    if (!req.query.pagenum || req.
query.pagenum <= 0)
        return res.sendResult(null,
            400, 'pagenum 参数错误');
    if (!req.query.pagesize || req.
query.pagesize <= 0)
        return res.sendResult(null,
            400, 'pagesize 参数错误');
    next();
}, function(req, res, next) {
    ManagerService.getAllUsers
    (conditions, function(err,
result) {
        if (err) return res.sendResult
        (null, 400, err);
        res.sendResult(result, 200, '获
        取成功');
    });
});
});

```

服务层实现:

在 services/ManagerService.js 中实现用户管理业务逻辑:

获取用户列表:

```

module.exports.getAllUsers =
function(params, cb) {
    var conditions = {};
    if (params.query) {
        conditions['columns']
        ['mg_name'] = orm.like('%' +
        params.query + '%');
    }

    dao.countByConditions
    ('ManagerModel', conditions,
function(err, count) {
        var offset = (params.pagenum
        1) * params.pagesize;
        conditions['offset'] = offset;
        conditions['limit'] = params.
        pagesize;
    });
}

```

```

        conditions['only'] = ['mg_id',
        'mg_name', 'mg_mobile',
        'mg_email', 'mg_state',
        'mg_time', 'role_id'];
        conditions['order'] =
        'mg_time';

        dao.list('ManagerModel',
        conditions, function(err,
        users) {
            cb(null, {total: count,
            users: users});
        });
    });
};

```

更新用户：

```

module.exports.updateUser = function
(id, params, cb) {
    params.mg_id = id;
    generateUserInfo(params)
        .then(checkUserName)
        .then(updateUserInfo)
        .then(function(user) {
            cb(null, user);
        })
        .catch(function(err) {
            cb(err);
        });
};

```

删除用户（软删除）：

```

module.exports.deleteUser = function
(id, cb) {
    dao.update('ManagerModel', id,
    {mg_state: 0}, function(err) {
        if (err) return cb(err);
        cb(null);
    });
};

```

数据验证：

检查用户名是否重复

验证必填字段

验证数据格式（手机号、邮箱等）

(2) 用户管理 API 接口测试截图

Home Workspaces API Network

POST http://localhost:8088/ GET http://127.0.0.1:8088/sy: +

No environment

http://127.0.0.1:8088/sysapi/users

GET http://127.0.0.1:8088/sysapi/users?pagenum=1&pagesize=10

Send

Docs Params Authorization Headers (7) Body Scripts Settings Cookies

Key	Value	Description
User-Agent	PostmanRuntime/7.51.0	
Accept	*/*	
Accept-Encoding	gzip, deflate, br	
Connection	keep-alive	
Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6I...	

Body 200 OK • 5 ms • 793 B

JSON Preview Visualize

```
20  "mg_time": 1677934269,
21  "role_id": 0,
22  "mg_mobile": "13518148888",
23  "mg_email": "2013976072@qq.com",
24  "mg_state": 0
25  },
26  {
27    "mg_id": 2,
28    "mg_name": "operator",
29    "mg_pwd": null,
30    "mg_time": 1677934269,
31    "role_id": 2,
32    "mg_mobile": "13518148888",
33    "mg_email": "11@qq.com",
34    "mg_state": 0
35  }
36  ],
37  "meta": {
38    "msg": "获取成功",
39    "status": 200
40  }
41  }
42  }
```

Files BETA

Cloud View Console Terminal Runner Vault

Home Workspaces API Network

POST http://localhost:8088/ GET http://127.0.0.1:8088/sy/ + No environment

http://127.0.0.1:8088/sysapi/users/1 Save Share

GET http://127.0.0.1:8088/sysapi/users/1 Send

Docs Params Authorization Headers (7) Body Scripts Settings Cookies

Key	Value	Description
☒ User-Agent	PostmanRuntime/7.51.0	
☒ Accept	*/*	
☒ Accept-Encoding	gzip, deflate, br	
☒ Connection	keep-alive	
☒ Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6I...	

Body 200 OK • 5 ms • 470 B

JSON Preview Visualize

```
1 {
2   "data": {
3     "mg_id": 1,
4     "mg_name": "admin",
5     "mg_pwd": "123456",
6     "mg_time": 1677934269,
7     "role_id": 0,
8     "mg_mobile": "13518148888",
9     "mg_email": "2013976072@qq.com",
10    "mg_state": 0
11  },
12  "meta": {
13    "msg": "获取成功",
14    "status": 200
15  }
16 }
```

Files BETA

Cloud View Console Terminal Runner Vault

Home Workspaces API Network

POST http://localhost:8088/ PUT http://127.0.0.1:8088/sy. +

No environment

http://127.0.0.1:8088/sysapi/users/2

PUT http://127.0.0.1:8088/sysapi/users/2 Send

Docs Params Authorization Headers (9) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beautify

```
1 {"mg_name": "newname", "mg_mobile": "13800138000"}
```

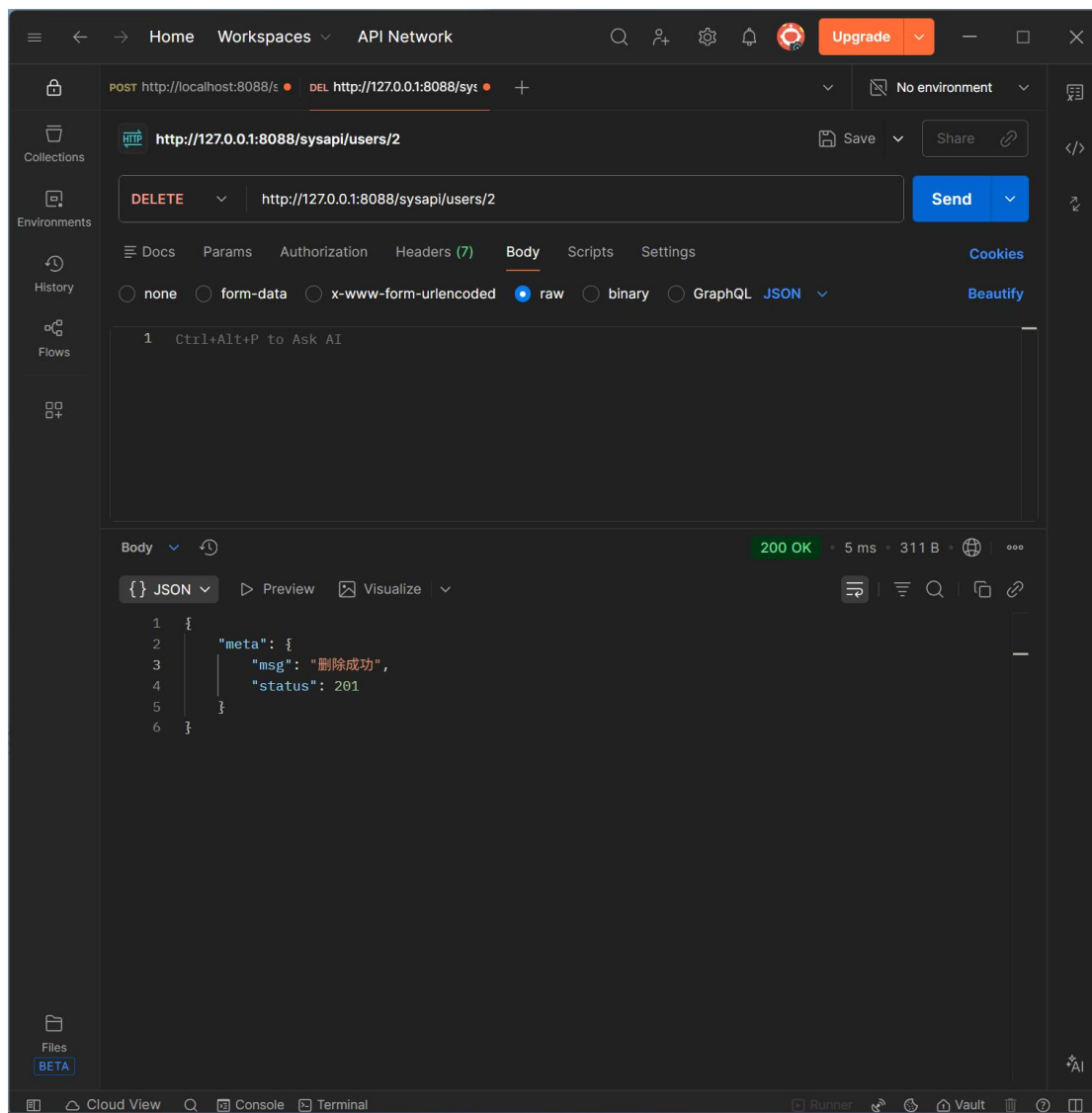
Body 200 OK 24 ms 525 B

JSON Preview Visualize

```
1 {
2   "data": {
3     "mg_id": 2,
4     "mg_name": "newname",
5     "mg_pwd": "$2y$10$I3ploVkaC4t51cC8x8nvh.QqVaNN2gITpnBSVJzgQkQL6fpRBr4C",
6     "mg_time": 1677934269,
7     "role_id": 2,
8     "mg_mobile": "13800138000",
9     "mg_email": "11@qq.com",
10    "mg_state": 0
11  },
12  "meta": {
13    "msg": "更新用户成功",
14    "status": 200
15  }
16 }
```

Files BETA

Cloud View Console Terminal Runner Vault



（二）前端 Vue 管理端

1. 登录页面

（1）登录页面源码

```
<template>

  <div class="container">

    <div class="loginbox">

      <div class="leftbox">

      </div>

      <div class="rightbox">

        <div class="logo">

        </div>

        <p>百果园管理后台</p>

        <div class="inputbox">
```

```


<elinput vmodel="account" placeholder="请输入账号" class="inputinner"/>
</div>

<div class="inputbox">

<elinput vmodel="password" placeholder="请输入密码" class="inputinner"/>
</div>

<elbutton type="primary" class="loginbtn" @click="login">登录</elbutton>
<elbutton @click="logout" class="loginbtn">退出</elbutton>
</div>

</div>

</div>

</template>
<script setup>
  import axios from "axios";
  import {ref} from 'vue'
  import {ElMessage} from 'elementplus'
  import {useRouter} from 'vuerouter'

  const router=useRouter()

  const account = ref()
  const password = ref()
  const login=async()=>{
    // 数据校验
    if (account.value == "" || password.value == "") {
      ElMessage.error("请输入用户名和密码");
      return;
    }
    let loginForm = {
      username: account.value,
      password: password.value,
    };
    try {
      const { data: res } = await axios.post(`http://127.0.0.1:8088/sysapi/login`,
        loginForm);
      console.log(res);
      if (res.meta.status !== 200) {
        ElMessage.error("登录失败");
        return
      }
      ElMessage.success("登录成功");
      window.localStorage.setItem("token", res.data.token);
      router.push('/admin/index')
    } catch (e) {

```

```

console.log("登录失败"+e);

ElMessage.error("网络错误或服务器异常");

}

}

const logout=async()=>{
//退出
window.sessionStorage.clear();
router.push('/login')
}

</script>

<style scoped>
.container{
width: 100vw;
height: 100vh;
backgroundcolor: #f3f7fa;
display: flex;
justifycontent: center;
alignitems: center;
}

.loginbox{
width: 1000px;
height: 600px;
backgroundcolor:#fff;
borderradius: 10px;
overflow: hidden;
display: flex;
}

.leftbox{
width: 424px;
backgroundcolor: #333;
}

.leftbox img{
width: 100%;
height: 100%;
}

.rightbox{
flex: 1;
display: flex;
flexdirection: column;
alignitems: center;
}

.logo{
width: 70px;
height: 70px;

```

```

    margin-top: 90px;
    border-radius: 50%;
    overflow: hidden;
  }

  .logo img{
    width: 100%;
    height: 100%;
  }

  .rightbox p{
    font-size: 24px;
    font-family: Microsoft YaHeiBold,Microsoft YaHei;
    font-weight: bold;
    color: #272727;
    margin-bottom: 40px;
  }

  .inputbox{
    width: 328px;
    height: 50px;
    background-color: #f1f2f7;
    border-radius: 25px;
    margin-bottom: 25px;
    display: flex;
    align-items: center;
  }

  .prefixicon{
    width: 23px;
    height: 23px;
    margin: 0 8px 0 22px;
  }

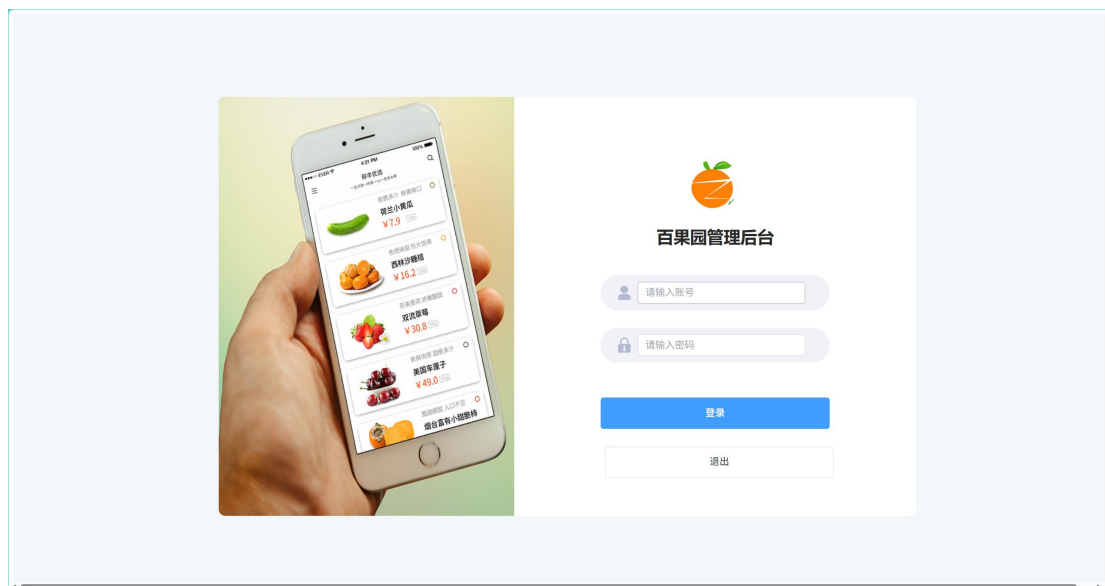
  .inputinner{
    width: 240px;
  }

  .loginbtn{
    width: 328px;
    height: 45px;
    margin-top: 25px;
    box-sizing: border-box;
  }
}

</style>

```

(2) 登录页面运行截图



2. 商品分类管理页面

(1) 商品分类页面源码

```
<template>

  <elcard class="boxcard">

    <!-- 搜索栏 -->

    <elrow :gutter="10">

      <elcol :span="6">

        <elinput vmodel="searchKeyword" placeholder="输入分类名称"></elinput>

      </elcol>

      <elcol :span="4">

        <elbutton type="primary" @click="handleSearch">搜索</elbutton>

      </elcol>

      <elcol :span="4">

        <elbutton type="primary" @click="showAddDialog">添加分类</elbutton>

      </elcol>

    </elrow>

    <eltable :data="catelist" style="width: 100%">

      <eltablecolumn label="分类编号" prop="cat_id" />

      <eltablecolumn label="分类名称" prop="cat_name" />

      <eltablecolumn label="分类级别" prop="cat_level" />

      <eltablecolumn label="操作" align="right">

        <template #default="scope">

          <elbutton size="small" @click="editCategory(scope.row.cat_id)">

            编辑</elbutton>

          >

          <elbutton

            size="small"

            type="danger"

            @click="deleteCategory(scope.row.cat_id)"

            >删除</elbutton>

        </template>

      </eltablecolumn>

    </eltable>

  </elcard>

</template>
```



```

        </template>

    </eltablecolumn>

</eltable>
</elcard>
<!-- 添加分类弹出层 begin -->
<el-dialog
    v-model="add_dialogVisible"
    title="添加分类"
    width="40%"
    :beforeclose="handleClose"
>
    <el-form
        ref="ruleFormRef"
        :model="addCateForm"
        status-icon
        :rules="rules"
        label-width="120px"
        class="demo-ruleForm"
    >
        <el-form-item label="分类名称: ">
            <el-input v-model="addCateForm.cat_name" autocomplete="off" />
        </el-form-item>
        <el-form-item label="父级分类: ">
            <el-select
                v-model="selectvalue"
                class="m2"
                placeholder="请选择"
                style="width: 240px"
            >
                <el-option
                    v-for="item in parentCateList"
                    :key="item.cat_id"
                    :label="item.cat_name"
                    :value="item.cat_id"
                />
            </el-select>
        </el-form-item>
        <el-form-item>
            <el-button type="primary" @click="addCate">确定</el-button>
            <el-button @click="cancelAddCate">取消</el-button>
        </el-form-item>
    </el-form>
</el-dialog>
<!-- 添加分类弹出层 end -->

```

```

<! 编辑分类弹出层 begin >
<el-dialog
  v-model="edit_dialogVisible"
  title="修改分类"
  width="40%"
  :beforeclose="handleClose"
>
  <el-form
    ref="ruleFormRef"
    :model="editCateForm"
    status-icon
    :rules="rules"
    label-width="120px"
    class="demo-ruleForm"
  >
    <el-form-item label="分类名称: ">
      <el-input v-model="editCateForm.cat_name" autocomplete="off" />
    </el-form-item>
    <el-form-item>
      <el-button type="primary" @click="editCate(editCateForm.cat_id)">
        确定
      </el-button>
      <el-button @click="edit_dialogVisible = false">取消</el-button>
    </el-form-item>
  </el-form>
</el-dialog>
<! 编辑分类弹出层 end >
</template>

<script setup>
import axios from 'axios'
import { onMounted, reactive, ref } from 'vue'
import { ElMessage } from 'element-plus'
const catelist = ref([])
const parentCatelist = ref([])
const add_dialogVisible = ref(false)
const selectvalue = ref()
const searchKeyword = ref('')
const addCateForm = reactive({
  cat_id: '',
  cat_name: '', // 将要添加的分类的名称
  cat_pid: 0, // 父级分类的 Id
  cat_level: 0 // 分类的等级，默认要添加的是 1 级分类
})

```

```

const edit_dialogVisible = ref(false)

const editCateForm = reactive({
  cat_id: '',
  cat_name: ''
})

// "添加分类"按钮事件
function showAddDialog () {
  add_dialogVisible.value = true
}

// 重置添加分类表单
const resetAddForm = () => {
  addCateForm.cat_name = ''
  selectvalue.value = null
}

```

```

// 取消添加分类
const cancelAddCate = () => {
  add_dialogVisible.value = false
  resetAddForm()
}

```

```

// 点击按钮，添加新的分类
const addCate = async () => {
  const token = localStorage.getItem('token')

```

```

// 根据父级分类计算分类级别
let catLevel = 1 // 默认一级分类
if (selectvalue.value) {
  // 如果选择了父级分类，查找父级分类的级别并+1
  const parentCategory = parentCateList.value.find(
    cate => cate.cat_id === selectvalue.value
  )
  if (parentCategory) {
    catLevel = parentCategory.cat_level + 1
  }
}

```

```

try {
  const { data: res } = await axios.post(
    `http://127.0.0.1:8088/sysapi/category`,
    {
      cat_pid: selectvalue.value || 0, // 父级分类的 Id
      cat_name: addCateForm.cat_name, // 将要添加的分类的名称
      cat_level: catLevel // 根据父级分类计算的分类级别

```

```

    },
    {
      headers: {
        Authorization: `${token}` // 最常见的格式
      }
    }
  )
}
if (res.meta.status !== 201) {
  ElMessage.error('添加分类失败! ')
  return
}
ElMessage.success('添加分类成功! ')
getCatelist()
add_dialogVisible.value = false
resetAddForm() // 添加成功后重置表单
} catch (error) {
  console.error('获取分类失败' + error)
}
}
//搜索分类函数
const handleSearch = () => {
  getCatelist(searchKeyword.value)
}

```

```

//获取类别列表
const getCatelist = async (keyword = '') => {
  const token = localStorage.getItem('token')
  try {
    const { data: res } = await axios.get(
      `http://127.0.0.1:8088/sysapi/category`,
      {
        params: {
          keyword: keyword
        },
        headers: {
          Authorization: `${token}` // 最常见的格式
        }
      }
    )
    console.log(res)
    catelist.value = res.data
    parentCatelist.value = res.data
  } catch (error) {
    console.error('获取分类失败' + error)
  }
}

```

```

    }
  }
  //生命周期钩子函数
  onMounted(() => {
    getCategoryList()
  })
  //删除分类
  const deleteCategory = async cat_id => {
    const token = localStorage.getItem('token')
    if (cat_id == '') {
      ElMessage.error('为获取需要删的 ID')
      return
    }
    try {
      const { data: res } = await axios.delete(
        `http://127.0.0.1:8088/sysapi/category/${cat_id}`,
        {
          headers: {
            Authorization: `${token}` // 最常见的格式
          }
        }
      )
      if (res.meta.status !== 400) {
        ElMessage.error('删除分类失败! ')
        return
      }
      ElMessage.success('删除分类成功! ')
      getCategoryList()
    } catch (error) {
      console.error('删除分类失败' + error)
    }
  }
  //编辑分类，渲染原始分类名称
  const editCategory = async cat_id => {
    edit_dialogVisible.value = true
    const token = localStorage.getItem('token')
    try {
      const { data: res } = await axios.get(
        `http://127.0.0.1:8088/sysapi/category/` + cat_id,
        {
          headers: {
            Authorization: `${token}` // 最常见的格式
          }
        }
      )
    }
  }

```

```

    )

    if (res.meta.status !== 200) {
      ElMessage.error('获取分类失败! ')
      return
    }

    editCateForm.cat_name = res.data.cat_name
    editCateForm.cat_id = res.data.cat_id
  } catch (error) {
    console.error('获取分类失败' + error)
  }
}

//点击按钮，编辑分类
const editCate = async cat_id => {
  const token = localStorage.getItem('token')
  try {
    const { data: res } = await axios.put(
      `http://127.0.0.1:8088/sysapi/category/` + cat_id,
      {
        cat_name: editCateForm.cat_name
      },
      {
        headers: {
          Authorization: `${token}` // 最常见的格式
        }
      }
    )

    if (res.meta.status !== 200) {
      ElMessage.error('更新分类失败! ')
      return
    }

    ElMessage.success('更新分类成功! ')
    getCateList()
    edit_dialogVisible.value = false
  } catch (error) {
    console.error('更新分类失败' + error)
  }
}
</script>

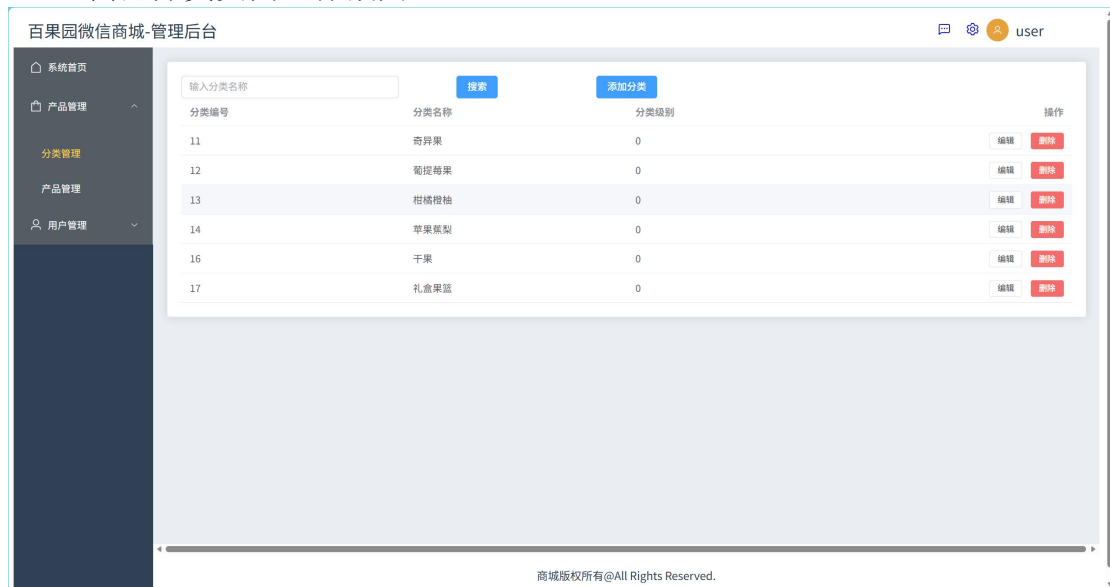
```

```

<style scoped>
.boxcard {
  width: 100%;
}
</style>

```

(2) 商品分类页面运行截图



3. 商品管理页面

(1) 商品管理页面源码

```
<template>

  <elcard class="boxcard">

    <template #header>

      <elrow :gutter="10">

        <elcol :span="4">

          <!-- 请选择商品分类 -->

          <elselect vmodel="value" class="m2" placeholder="请选择商品分类">

            <eloption

              vfor="item in options"

              :key="item.value"

              :label="item.label"

              :value="item.value"

            />

          </elselect>

        </elcol>

        <elcol :span="6">

          <!-- 输入商品名称 -->

          <elinput vmodel="queryInfo.query" placeholder="输入商品名称" />

        </elcol>

        <elcol :span="4">

          <elbutton type="primary" @click="getGoodsList">搜索</elbutton>

        </elcol>

        <elcol :span="4">

          <elbutton type="primary" @click="goAddProduct">添加商品</elbutton>

        </elcol>

      </elrow>

    </template>

  </elcard>

</template>
```

```

<eltable :data="goods" style="width: 100%">

  <eltablecolumn label="商品名称" prop="goods_name" />

  <eltablecolumn label="商品分类" prop="cat_id" />

  <eltablecolumn label="商品数量" prop="goods_number" />

  <eltablecolumn label="商品价格" prop="goods_price" />

  <eltablecolumn align="right" label="操作">

    <template #default="scope">

      <elbutton size="small" @click="goUpdateProduct(scope.row.goods_id)"

        >编辑</elbutton

      >

      <elbutton

        size="small"

        type="danger"

        @click="removeById(scope.row.goods_id)"

        >删除</elbutton

      >

    </template>

  </eltablecolumn>

</eltable>

<!-- 分页 -->

<elpagination

  vmodel:currentpage="queryInfo.pagenum"

  vmodel:pagesize="queryInfo.pagesize"

  :pagesizes="[5, 10, 15, 20]"

  :small="small"

  :disabled="disabled"

  :background="background"

  layout="total, sizes, prev, pager, next, jumper"

  :total="total"

  @sizechange="handleSizeChange"

  @currentchange="handleCurrentChange"

/>

</elcard>
</template>

<script setup>
import axios from 'axios'

import { onMounted, reactive, ref } from 'vue'

import { useRouter } from 'vuerouter'

import { ElMessage } from 'elementplus'

const router = useRouter()

const goods = ref([])

const options = ref([]) // 用于存储分类数据

const value = ref('') // 分类选择器的值

```



```

const queryInfo = reactive({
  query: '',
  pagenum: 1,
  pagesize: 10
})

const total = ref(0)

// 获取分类列表

const getCategoryList = async () => {
  const token = localStorage.getItem('token')
  try {
    const { data: res } = await axios.get(
      `http://127.0.0.1:8088/sysapi/category`,
      {
        headers: {
          Authorization: token
        }
      }
    )
    if (res.meta.status === 200) {
      // 将分类数据转换为 select 组件需要的格式
      options.value = res.data.map(item => ({
        label: item.cat_name,
        value: item.cat_id
      })))
    }
  } catch (error) {
    console.error('获取分类列表失败' + error)
    ElMessage.error('获取分类列表失败')
  }
}

```

```

//获取商品列表

const getGoodsList = async () => {
  const token = localStorage.getItem('token')
  try {
    const { data: res } = await axios.get(
      `http://127.0.0.1:8088/sysapi/goods`,
      {
        params: queryInfo,
        headers: {
          Authorization: token // 直接使用 token，因为后端已经在 token 中添加了 Bearer 前缀
        }
      }
    )
  }
}

```

```

    console.log(res)

    goods.value = res.data.goods
    total.value = res.data.total
  } catch (error) {
    console.error('获取商品失败' + error)
  }
}

```

```

// 处理分页大小变化
const handleSizeChange = newSize => {
  queryInfo.pagesize = newSize
  getGoodsList()
}

```

```

// 处理当前页码变化
const handleCurrentChange = newPage => {
  queryInfo.pagenum = newPage
  getGoodsList()
}

//生命周期钩子函数
onMounted(async () => {
  await getCategoryList() // 先获取分类列表
  getGoodsList() // 再获取商品列表
})

function goAddProduct () {
  router.push('/admin/addproduct')
}

```

```

// 跳转到编辑商品页面
function goUpdateProduct (goods_id) {
  router.push(`/admin/updateproduct/${goods_id}`)
}

//删除商品
const removeById = async goods_id => {
  const token = localStorage.getItem('token')
  if (goods_id == '') {
    ElMessage.error('未获取需要删的 ID')
    return
  }
  try {
    const { data: res } = await axios.delete(
      `http://127.0.0.1:8088/sysapi/goods/${goods_id}`,
      {
        headers: {

```

```

        Authorization: token // 直接使用 token，因为后端已经在 token 中添加了 Bearer 前缀
    }
}
)

if (res.meta.status !== 201) {
    ElMessage.error('删除商品失败! ')
    return
}

ElMessage.success('删除商品成功! ')
getGoodsList()
} catch (error) {
    console.error('删除分类失败' + error)
}
}
}
</script>

```

```
<style scoped></style>
```

```

<template>
  <!-- 卡片区域 -->
  <elcard>
    <!-- 提示区域 -->
    <elalert title="添加商品信息" type="info" center showicon />
    <!-- 步骤条区域 -->
    <elsteps :active="activeIndex" :finishstatus="success">
      <elstep title="基本信息" />
      <elstep title="商品图片" />
      <elstep title="商品详情" />
      <elstep title="完成" />
    </elsteps>
    <!-- tab 栏区域 -->
    <elform
      ref="formRef"
      :model="addForm"
      :rules="rules"
      labelwidth="120px"
      class="demodynamic"
      style="margin-top: 30px"
    >
      <eltabs :tabposition="tabPosition" class="demotabs">
        <!-- 第一个选项卡 -->
        <eltabpanel label="基本信息">
          <elFormItem label="商品名称" prop="goods_name">

```

```

        <elinput vmodel="addForm.goods_name"></elinput>
    </elformitem>
    <elformitem label="商品价格" prop="goods_price">
        <elinput
            vmodel.number="addForm.goods_price"
            type="number"
        ></elinput>
    </elformitem>
    <elformitem label="商品重量" prop="goods_weight">
        <elinput
            vmodel.number="addForm.goods_weight"
            type="number"
        ></elinput>
    </elformitem>
    <elformitem label="商品数量" prop="goods_number">
        <elinput
            vmodel.number="addForm.goods_number"
            type="number"
        ></elinput>
    </elformitem>
    <elformitem label="商品分类" prop="goods_cat">
        <elselect
            vmodel="addForm.goods_cat"
            class="m2"
            placeholder="请选择"
            style="width: 240px"
        >
            <eloption
                vfor="item in catelist"
                :key="item.cat_id"
                :label="item.cat_name"
                :value="item.cat_id"
            />
        </elselect>
    </elformitem>
</eltabpanel>
<! 第二个选项卡 >
<eltabpanel label="商品图片">
    <elupload
        class="uploaddemo"
        :httprequest="uploadImage"
        :filelist="fileList"
        action="#"
        :onpreview="handlePreview"
    >

```

```

      :onremove="handleRemove"

      listtype="picture"
    >

      <elbutton type="primary">点击上传</elbutton>

    </elupload>
  </eltabpanel>
<!-- 第三个选项卡 -->
<eltabpanel label="商品详情">

  <!-- 富文本编辑器组件 -->
  <QuillEditor

    theme="snow"

    style="height: 300px"

    vmodel:content="addForm.goods_introduce"

    contentType="html"

    :toolbar="[

      ['bold', 'italic', 'underline', 'strike'],

      ['blockquote', 'codeblock'],

      ['link', 'image', 'video', 'formula'],

      [{ header: 1 }, { header: 2 }],

      [{ list: 'ordered' }, { list: 'bullet' }, { list: 'check' }],

      [{ script: 'sub' }, { script: 'super' }],

      [{ indent: '1' }, { indent: '+1' }],

      [{ direction: 'rtl' }],

      [{ size: ['small', false, 'large', 'huge'] }],

      [{ header: [1, 2, 3, 4, 5, 6, false] }],

      [{ color: [] }, { background: [] }],

      [{ font: [] }],

      [{ align: [] }],

      ['clean']

    ]"

  />

  <!-- 添加商品的按钮 -->
  <elbutton

    type="primary"

    class="btnAdd"

    @click="add"

    style="

      margin-top: 20px;

      display: block;

      margin-left: auto;

      margin-right: auto;

    "

  >添加商品</elbutton>
</eltabpanel>

```

```

        </eltabpane>

      </eltabs>

    </elform>

  </elcard>
</template>
<script setup>
import axios from 'axios'

import { onMounted, reactive, ref } from 'vue'
import { ElMessage } from 'elementplus'
import { useRouter } from 'vuerouter'
import { QuillEditor } from '@vueup/vuequill'
import '@vueup/vuequill/dist/vuequill.snow.css'

const router = useRouter()

// 表单引用
const formRef = ref(null)

```

```

// 步骤条激活索引
const activeIndex = ref('0')

```

```

// 父分类列表
const parentCateList = ref([])

```

```

// 文件列表
const fileList = ref([])

```

```

// 表单验证规则
const rules = ref({
  goods_name: [
    { required: true, message: '请输入商品名称', trigger: 'blur' },
    {
      min: 1,
      max: 50,
      message: '商品名称长度在 1 到 50 个字符',
      trigger: 'blur'
    }
  ],
  goods_price: [
    { required: true, message: '请输入商品价格', trigger: ['blur', 'change'] },
    {
      validator: (rule, value, callback) => {
        if (value === null || value === undefined || value === '') {
          callback(new Error('请输入商品价格'))
        } else if (typeof value !== 'number') {

```

```

        callback(new Error('请输入有效的数字'))
    } else if (value <= 0) {
        callback(new Error('商品价格必须大于 0'))
    } else {
        callback()
    }
},
trigger: ['blur', 'change']
}
],
goods_weight: [
{ required: true, message: '请输入商品重量', trigger: ['blur', 'change'] },
{
    validator: (rule, value, callback) => {
        if (value === null || value === undefined || value === '') {
            callback(new Error('请输入商品重量'))
        } else if (typeof value !== 'number') {
            callback(new Error('请输入有效的数字'))
        } else if (value <= 0) {
            callback(new Error('商品重量必须大于 0'))
        } else {
            callback()
        }
    },
    trigger: ['blur', 'change']
}
],
goods_number: [
{ required: true, message: '请输入商品数量', trigger: ['blur', 'change'] },
{
    validator: (rule, value, callback) => {
        if (value === null || value === undefined || value === '') {
            callback(new Error('请输入商品数量'))
        } else if (typeof value !== 'number') {
            callback(new Error('请输入有效的数字'))
        } else if (value <= 0) {
            callback(new Error('商品数量必须大于 0'))
        } else if (!Number.isInteger(value)) {
            callback(new Error('商品数量必须为整数'))
        } else {
            callback()
        }
    },
    trigger: ['blur', 'change']
}
]

```

```

    }
  ],
  goods_cat: [{ required: true, message: '请选择商品分类', trigger: 'change' }]
}))

```

```

const addForm = reactive({
  goods_name: '',
  goods_price: null,
  goods_weight: null,
  goods_number: null,
  // 商品所属的分类
  goods_cat: '',
  // 图片的数组
  pics: [],
  // 商品的详情描述
  goods_introduce: ''
})

//获取类别列表
const catelist = ref([])
const getCateList = async () => {
  const token = localStorage.getItem('token')
  try {
    const { data: res } = await axios.get(
      `http://127.0.0.1:8088/sysapi/category`,
      {
        headers: {
          Authorization: `${token}` // 最常见的格式
        }
      }
    )
    console.log(res)
    catelist.value = res.data
    parentCateList.value = res.data
  } catch (error) {
    console.error('获取分类失败' + error)
  }
}

//Tabs 标签页的方向
const tabPosition = ref('left')

//生命周期钩子函数
onMounted(() => {
  getCateList()
})

```



```

//图片上传

const uploadImage = async option => {

  const { file, onSuccess, onError } = option

  const formData = new FormData()

  const token = localStorage.getItem('token')

  formData.append('file', file)

  try {

    const { data: res } = await axios.post(

      'http://127.0.0.1:8088/sysapi/upload',

      formData,

      {

        headers: {

          Authorization: `${token}` // 最常见的格式

        }

      }

    )

    console.log(res)

    const uploadedFile = {

      uid: file.uid,

      name: file.name,

      status: 'success',

      url: res.data.tmp_path

    }

    fileList.value.push(uploadedFile)

    addForm.pics.push({ pic: res.data.tmp_path })

    onSuccess && onSuccess()

  } catch (err) {

    onError(err)

  }

}

```

```

// 图片预览函数

const handlePreview = file => {

  console.log(file)

}

```

```

// 图片移除函数

const handleRemove = file => {

  // 从显示列表中移除

  const index = fileList.value.findIndex(item => item.uid === file.uid)

  if (index !== 1) {

    fileList.value.splice(index, 1)

  }

  // 从数据列表中移除

```

```

const dataIndex = addForm.pics.findIndex(item => item.pic === file.url)
if (dataIndex !== 1) {
  addForm.pics.splice(dataIndex, 1)
}

return true
}

```

```

// 提交添加商品的表单
const add = async () => {
  if (!formRef.value) return
  try {
    await formRef.value.validate()

```

```

// 准备提交数据，确保数据类型正确
const submitData = {
  ...addForm,
  goods_price: Number(addForm.goods_price),
  goods_weight: Number(addForm.goods_weight),
  goods_number: Number(addForm.goods_number),
  // 确保 goods_cat 是字符串类型
  goods_cat: String(addForm.goods_cat)
}

```

```

console.log('表单验证通过，提交数据:', submitData)
const token = localStorage.getItem('token')
console.log('token:', token)
const { data: res } = await axios.post(
  'http://127.0.0.1:8088/sysapi/goods',
  submitData,
  {
    headers: {
      Authorization: `${token}`
    }
  }
)
console.log('API 响应:', res)
if (res.meta.status === 201) {
  ElMessage.success('添加商品成功')
  router.push('/admin/productlist')
} else {
  ElMessage.error(res.meta.msg || '添加商品失败')
}
} catch (error) {
  console.error('添加商品失败:', error)

```

```

    if (error.response) {
      console.error('响应错误:', error.response)
      ElMessage.error(
        error.response.data.meta?.msg ||
        error.response.data?.message ||
        '添加商品失败'
      )
    } else if (error.request) {
      console.error('请求错误:', error.request)
      ElMessage.error('服务器无响应, 请检查网络连接')
    } else {
      console.error('请求配置错误:', error.message)
      ElMessage.error('网络错误, 请稍后重试')
    }
  }
}
</script>
<style scoped>
.demotabs > .eltabs__content {
  padding: 32px;
  color: #6b778c;
  font-size: 32px;
  font-weight: 600;
}
</style>

```

(2) 商品管理页面运行截图

百果园微信商城-管理后台

system

user

系统首页

产品管理

分类管理

产品管理

用户管理

请选择商品分类

输入商品名称

搜索

添加商品

商品名称	商品分类	商品数量	商品价格	操作
43	0	34	34	编辑 删除
飞弹发射	0	234	34	编辑 删除
34	0	432	32	编辑 删除
新鲜苹果	0	100	28	编辑 删除
汉源车厘子	0	100	129	编辑 删除
妃子笑荔枝	0	100	99.8	编辑 删除

Total 6

10/page

< 1 >

Go to 1

商城版权所有@All Rights Reserved.

百果园微信商城-管理后台

system

user

系统首页

产品管理

分类管理

产品管理

用户管理

添加商品信息

1

2

3

4

基本信息

商品图片

商品详情

完成

基本信息

商品图片

商品详情

* 商品名称

* 商品价格

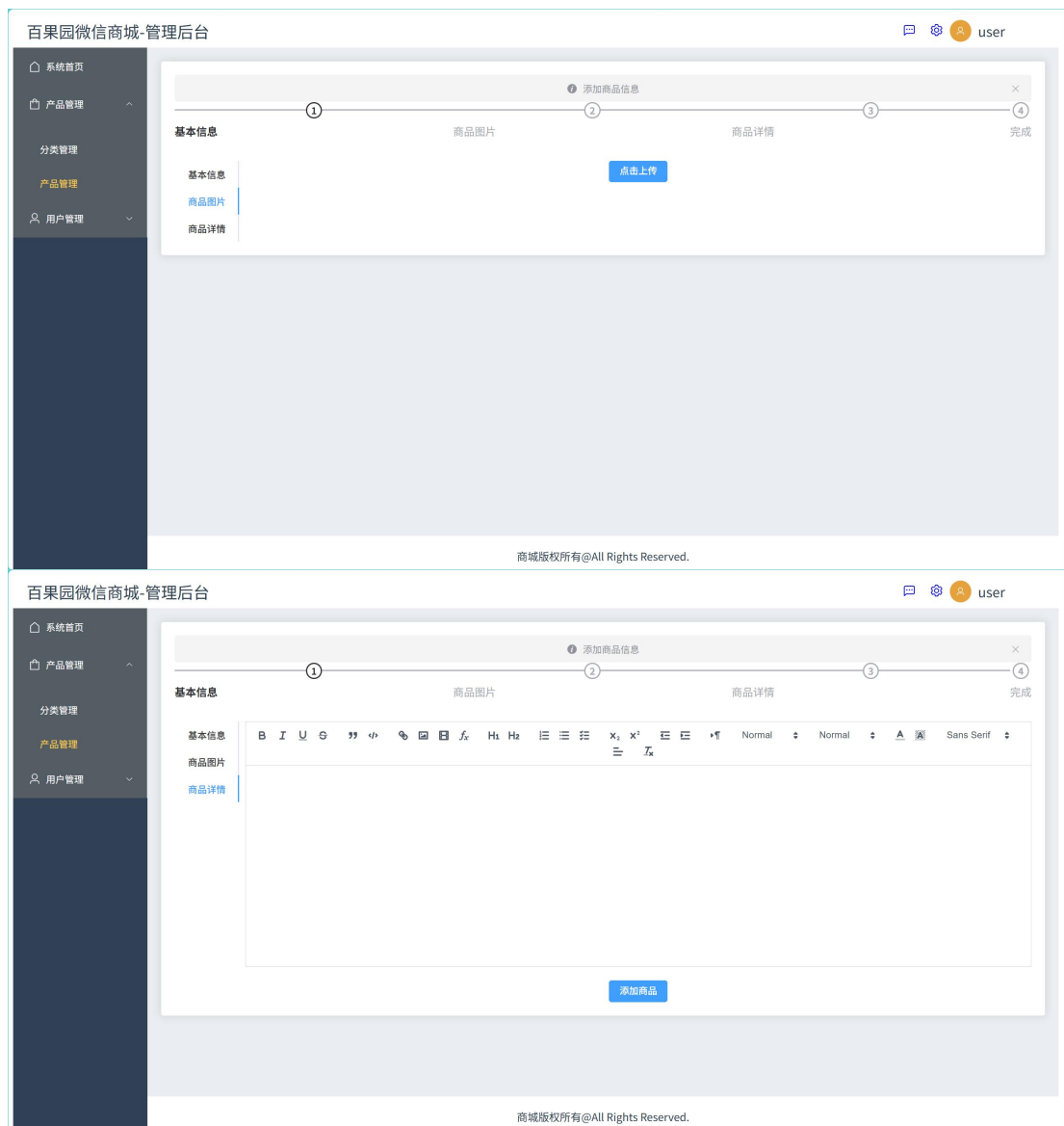
* 商品重量

* 商品数量

* 商品分类

请选择

商城版权所有@All Rights Reserved.



4. 用户管理页面

(1) 用户管理页面源码

```
<template>

  <elcard class="boxcard">

    <template #header>

      <elrow :gutter="10">

        <elcol :span="8">

          <!-- 输入用户名 -->

          <elinput vmodel="queryInfo.query" placeholder="输入用户名" />

        </elcol>

        <elcol :span="4">

          <elbutton type="primary" @click="getUserList">搜索</elbutton>

        </elcol>

      </elrow>

    </template>

    <eltable :data="users" style="width: 100%">

      <eltablecolumn label="用户 ID" prop="mg_id" width="80" />

    </eltable>

  </elcard>

</template>
```

```

<eltablecolumn label="用户名" prop="mg_name" />
<eltablecolumn label="手机" prop="mg_mobile" />
<eltablecolumn label="邮箱" prop="mg_email" />
<eltablecolumn label="角色" prop="role_id" />
<eltablecolumn label="状态" prop="mg_state">
  <template #default="scope">
    <elswitch
      vmodel="scope.row.mg_state"
      :activevalue="1"
      :inactivevalue="0"
      @change="handleStatusChange(scope.row)"
    />
  </template>
</eltablecolumn>
<eltablecolumn label="创建时间" prop="mg_time">
  <template #default="scope">
    {{ formatDate(scope.row.mg_time) }}
  </template>
</eltablecolumn>
<eltablecolumn align="right" label="操作">
  <template #default="scope">
    <elbutton size="small" @click="goUpdateUser(scope.row.mg_id)">
      编辑
    </elbutton>
    <elbutton
      size="small"
      type="danger"
      @click="removeById(scope.row.mg_id)"
    >
      删除
    </elbutton>
  </template>
</eltablecolumn>
</eltable>
<!-- 分页 -->
<elpagination
  vmodel:currentpage="queryInfo.pagenum"
  vmodel:pagesize="queryInfo.pagesize"
  :pagesizes="[5, 10, 15, 20]"
  :background="true"
  layout="total, sizes, prev, pager, next, jumper"
  :total="total"
  @sizechange="handleSizeChange"
  @currentchange="handleCurrentChange"
-->

```

```

    />

  </elcard>
</template>

<script setup>
import axios from 'axios'
import { onMounted, reactive, ref } from 'vue'
import { useRouter } from 'vuerouter'
import { ElMessage } from 'elementplus'

```

```

const router = useRouter()
const users = ref([])
const queryInfo = reactive({
  query: '',
  pagenum: 1,
  pagesize: 10
})
const total = ref(0)

```

```

// 格式化时间
const formatDate = timestamp => {
  if (!timestamp) return ''
  const date = new Date(timestamp)
  return date.toLocaleString()
}

```

```

// 获取用户列表
const getUserList = async () => {
  const token = localStorage.getItem('token')
  try {
    const { data: res } = await axios.get(
      `http://127.0.0.1:8088/sysapi/users`,
      {
        params: queryInfo,
        headers: {
          Authorization: token
        }
      }
    )
  }
}

```

```

if (res.meta.status !== 200) {
  ElMessage.error('获取用户列表失败')
  return
}

```

```

    users.value = res.data.users
    total.value = res.data.total
  } catch (error) {
    console.error('获取用户列表失败' + error)
    ElMessage.error('获取用户列表失败')
  }
}

```

```

// 处理分页大小变化
const handleSizeChange = newSize => {
  queryInfo.pagesize = newSize
  getUserList()
}

```

```

// 处理当前页码变化
const handleCurrentChange = newPage => {
  queryInfo.pagenum = newPage
  getUserList()
}

```

```

// 处理用户状态变更
const handleStatusChange = async user => {
  const token = localStorage.getItem('token')
  try {
    const { data: res } = await axios.put(
      `http://127.0.0.1:8088/sysapi/users/${user.mg_id}`,
      {
        mg_state: user.mg_state
      },
      {
        headers: {
          Authorization: token
        }
      }
    )
  }
}

```

```

if (res.meta.status !== 200) {
  ElMessage.error('更新用户状态失败')
  // 恢复原来的状态
  user.mg_state = user.mg_state === 1 ? 0 : 1
  return
}

```



```

    ElMessage.success('更新用户状态成功')
  } catch (error) {
    console.error('更新用户状态失败' + error)
    ElMessage.error('更新用户状态失败')
    // 恢复原来的状态
    user.mg_state = user.mg_state === 1 ? 0 : 1
  }
}

```

```

// 跳转到编辑用户页面
function goUpdateUser (mg_id) {
  router.push(`/admin/updateuser/${mg_id}`)
}

```

```

// 删除用户
const removeById = async mg_id => {
  const token = localStorage.getItem('token')
  if (!mg_id) {
    ElMessage.error('未获取需要删除的用户 ID')
    return
  }
}

```

```

try {
  const { data: res } = await axios.delete(
    `http://127.0.0.1:8088/sysapi/users/${mg_id}`,
    {
      headers: {
        Authorization: token
      }
    }
  )
}
)

```

```

if (res.meta.status !== 201) {
  ElMessage.error('删除用户失败! ')
  return
}

```

```

ElMessage.success('删除用户成功! ')
getUserList()
} catch (error) {
  console.error('删除用户失败' + error)
  ElMessage.error('删除用户失败! ')
}
}

```

```
// 生命周期钩子函数
```

```
onMounted(() => {  
  getUserList()  
})  
</script>
```

```
<style scoped></style>
```

```
<template>  
  <elcard>  
    <elalert title="编辑用户信息" type="info" center showicon />  
    <elform  
      ref="formRef"  
      :model="updateForm"  
      :rules="rules"  
      labelwidth="120px"  
      style="margin-top: 30px"  
    >  
      <elformitem label="用户名" prop="mg_name">  
        <elinput vmodel="updateForm.mg_name"></elinput>  
      </elformitem>  
      <elformitem label="密码（可选）" prop="mg_pwd">  
        <elinput  
          vmodel="updateForm.mg_pwd"  
          type="password"  
          placeholder="不修改请留空"  
        ></elinput>  
      </elformitem>  
      <elformitem label="手机" prop="mg_mobile">  
        <elinput vmodel="updateForm.mg_mobile"></elinput>  
      </elformitem>  
      <elformitem label="邮箱" prop="mg_email">  
        <elinput vmodel="updateForm.mg_email"></elinput>  
      </elformitem>  
      <elformitem label="角色 ID" prop="role_id">  
        <elinput vmodel.number="updateForm.role_id" type="number"></elinput>  
      </elformitem>  
      <elformitem label="状态" prop="mg_state">  
        <elswitch  
          vmodel="updateForm.mg_state"  
          activevalue="1"
```

```

        inactivevalue="0"

      </elswitch>

    </elformitem>

    <elformitem>

      <elbutton type="primary" @click="update">更新用户</elbutton>

      <elbutton @click="resetForm">重置</elbutton>

      <elbutton @click="goBack">取消</elbutton>

    </elformitem>

  </elform>

</elcard>

</template>

<script setup>
import axios from 'axios'
import { reactive, ref, onMounted } from 'vue'
import { ElMessage } from 'elementplus'
import { useRouter, useRoute } from 'vuerouter'

const router = useRouter()
const route = useRoute()
const formRef = ref(null)

```

```

const updateForm = reactive({
  mg_id: '',
  mg_name: '',
  mg_pwd: '',
  mg_mobile: '',
  mg_email: '',
  role_id: null,
  mg_state: '1'
})

```

```

const rules = reactive({
  mg_name: [
    { required: true, message: '请输入用户名', trigger: 'blur' },
    { min: 3, max: 20, message: '用户名长度在 3 到 20 个字符', trigger: 'blur' }
  ],
  mg_pwd: [
    { min: 6, max: 20, message: '密码长度在 6 到 20 个字符', trigger: 'blur' }
  ],
  mg_mobile: [
    { required: true, message: '请输入手机号', trigger: 'blur' },
    {
      pattern: /^1[39]\d{9}$/,
      message: '请输入有效的手机号码',

```

```

      trigger: 'blur'
    }
  ],
  mg_email: [
    { required: true, message: '请输入邮箱', trigger: 'blur' },
    {
      pattern: /^[azAZ09_]+@[azAZ09_]+(\.[azAZ09_]+)+$/ ,
      message: '请输入有效的邮箱地址',
      trigger: 'blur'
    }
  ],
  role_id: [
    { required: true, message: '请输入角色 ID', trigger: ['blur', 'change'] },
    {
      type: 'number',
      min: 0,
      message: '角色 ID 必须为非负整数',
      trigger: ['blur', 'change']
    }
  ]
})

```

```

// 获取用户详情
const getUserDetail = async () => {
  const token = localStorage.getItem('token')
  const userId = route.params.id

```

```

try {
  const { data: res } = await axios.get(
    `http://127.0.0.1:8088/sysapi/users/${userId}`,
    {
      headers: {
        Authorization: token
      }
    }
  )
}

```

```

if (res.meta.status === 200) {
  const user = res.data
  updateForm.mg_id = user.mg_id
  updateForm.mg_name = user.mg_name
  updateForm.mg_mobile = user.mg_mobile
  updateForm.mg_email = user.mg_email
  updateForm.role_id = Number(user.role_id)

```

```

    updateForm.mg_state = user.mg_state
  } else {
    ElMessage.error(res.meta.msg || '获取用户详情失败')
  }
} catch (error) {
  console.error('获取用户详情失败:', error)
  ElMessage.error('网络错误, 请稍后重试')
}
}
}

```

```

// 更新用户
const update = async () => {
  if (!formRef.value) return

```

```

  try {
    await formRef.value.validate()

```

```

    // 准备提交数据, 移除空密码
    const submitData = { ...updateForm }
    if (!submitData.mg_pwd) {
      delete submitData.mg_pwd
    }

```

```

    const token = localStorage.getItem('token')
    const { data: res } = await axios.put(
      `http://127.0.0.1:8088/sysapi/users/${updateForm.mg_id}`,
      submitData,
      {
        headers: {
          Authorization: token
        }
      }
    )

```

```

    if (res.meta.status === 200) {
      ElMessage.success('更新用户成功')
      router.push('/admin/userlist')
    } else {
      ElMessage.error(res.meta.msg || '更新用户失败')
    }
  } catch (error) {
    console.error('更新用户失败:', error)
    if (error.response) {
      ElMessage.error(error.response.data.meta?.msg || '更新用户失败')
    }
  }
}

```

```

    } else {
      ElMessage.error('网络错误, 请稍后重试')
    }
  }
}
}

```

```

// 重置表单
const resetForm = () => {
  if (formRef.value) {
    formRef.value.resetFields()
  }
}

```

```

// 返回用户列表
const goBack = () => {
  router.push('/admin/userlist')
}

```

```

// 生命周期钩子函数
onMounted(() => {
  getUserDetail()
})
</script>
<style scoped></style>

```

(2) 用户管理页面运行截图

百果园微信商城-管理后台

user

输入用户名 搜索

用户ID	用户名	手机	邮箱	角色	状态	创建时间	操作
3	123456	17310828917	3043259845@qq.com	0	☐	1970/1/21 08:51:22	编辑 删除
1	admin	13518148888	2013976072@qq.com	0	☐	1970/1/20 18:05:34	编辑 删除
2	operator	13518148888	11@qq.com	2	☐	1970/1/20 18:05:34	编辑 删除

Total 3 10/page < 1 > Go to 1

商城版权所有@All Rights Reserved.

三. 收获和体会

通过本次全栈开发实训，我完成了基于 Vue3 + Express 的商品展示管理系统的开发，在实践中收获颇丰：

1. 技术能力提升

前端技术方面：

- 深入掌握了 Vue 3 Composition API 的使用，包括 ref、reactive、onMounted 等核心 API
- 熟练运用 Vue Router 进行路由管理，实现了页面间的跳转和参数传递
- 掌握了 Element Plus 组件库的使用，快速构建了美观的管理界面
- 学会了使用 Axios 进行 HTTP 请求，处理异步数据交互
- 掌握了富文本编辑器 Vue Quill 的集成和使用

后端技术方面：

- 熟练使用 Express 框架构建 RESTful API，理解了中间件的工作机制
- 掌握了 ORM2 对象关系映射框架，简化了数据库操作
- 学会了使用 Passport.js 进行用户认证，理解了 JWT Token 的工作原理
- 掌握了 Multer 文件上传中间件的使用，实现了图片上传功能
- 理解了 Promise 和 async/await 异步编程模式

数据库方面：

- 掌握了 MySQL 数据库的基本操作和表结构设计
- 理解了 ORM 框架如何将对象映射到数据库表
- 学会了使用分页查询提高数据加载效率

2. 项目架构理解

通过本次项目，我深刻理解了前后端分离架构的优势：

- 前端专注于用户界面和交互体验
- 后端专注于业务逻辑和数据处理
- 通过 RESTful API 进行数据交互，职责清晰
- 便于团队协作开发和维护

同时理解了分层架构的重要性：

- 路由层：处理 HTTP 请求和参数验证
- 服务层：实现业务逻辑
- DAO 层：封装数据库操作
- 模型层：定义数据结构

3. 问题解决能力

在开发过程中遇到了诸多问题，通过查阅文档、调试代码，我学会了：

- 如何使用浏览器开发者工具调试前端代码
- 如何使用 Postman 测试 API 接口
- 如何查看服务器日志定位后端错误

如何处理跨域问题

如何处理文件上传和图片预览

如何实现分页查询

4. 代码规范和最佳实践

学会了使用 ESLint 进行代码规范检查

理解了模块化开发的重要性

学会了封装可复用的组件和函数

掌握了错误处理和用户提示的最佳实践

5. 团队协作意识

虽然本次实训是个人项目，但通过代码的组织和文档的编写，我认识到：

代码注释和文档的重要性

版本控制工具（如 Git）在团队协作中的作用

接口文档对前后端协作的重要性